Module 1

SystemVerilog and Verification Basics

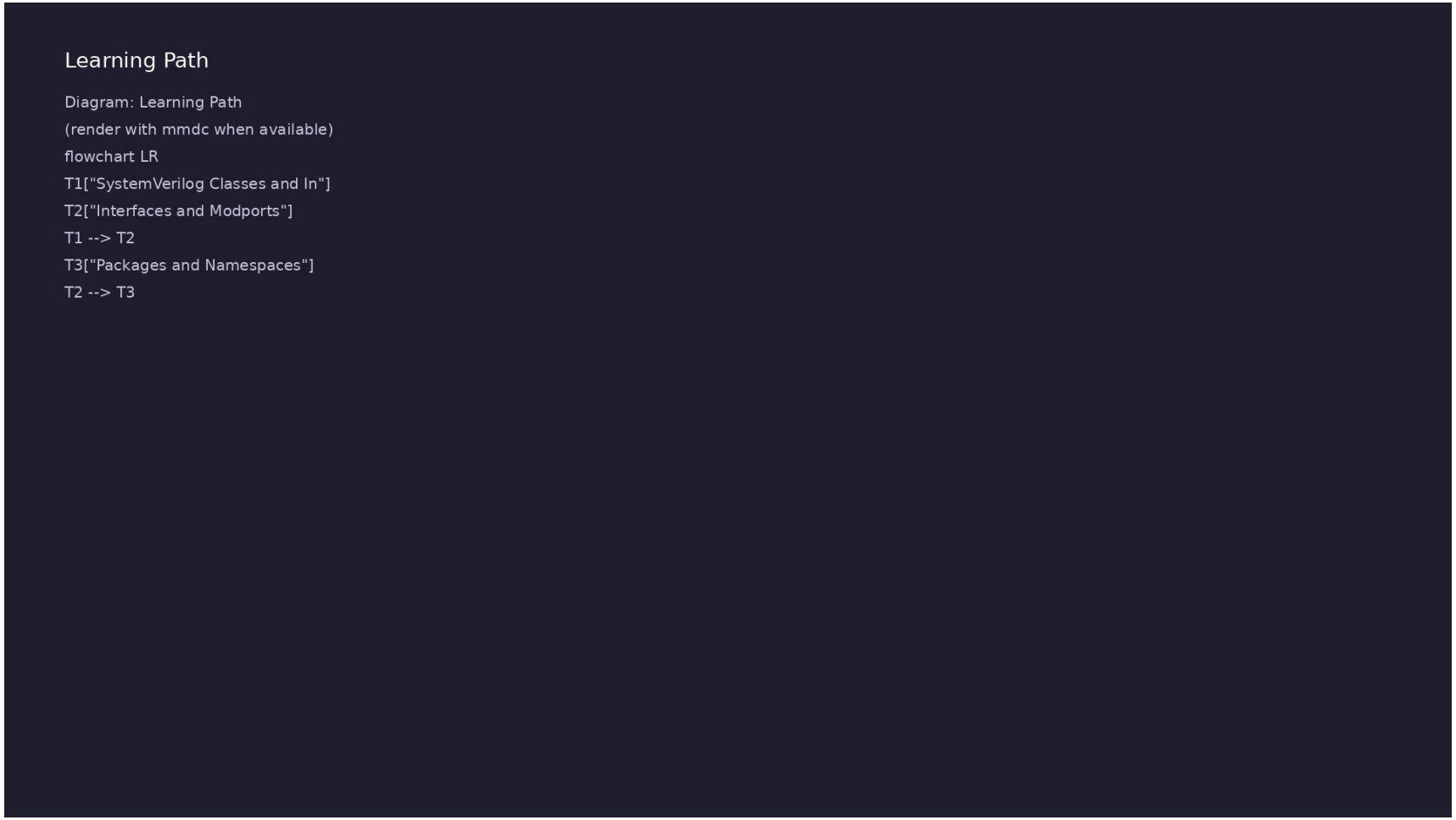
Learning objectives

- Understand SystemVerilog for verification and verification fundamentals

Prerequisites

- See module README

Learning path



Understand SystemVerilog for verification and verification fundamentals

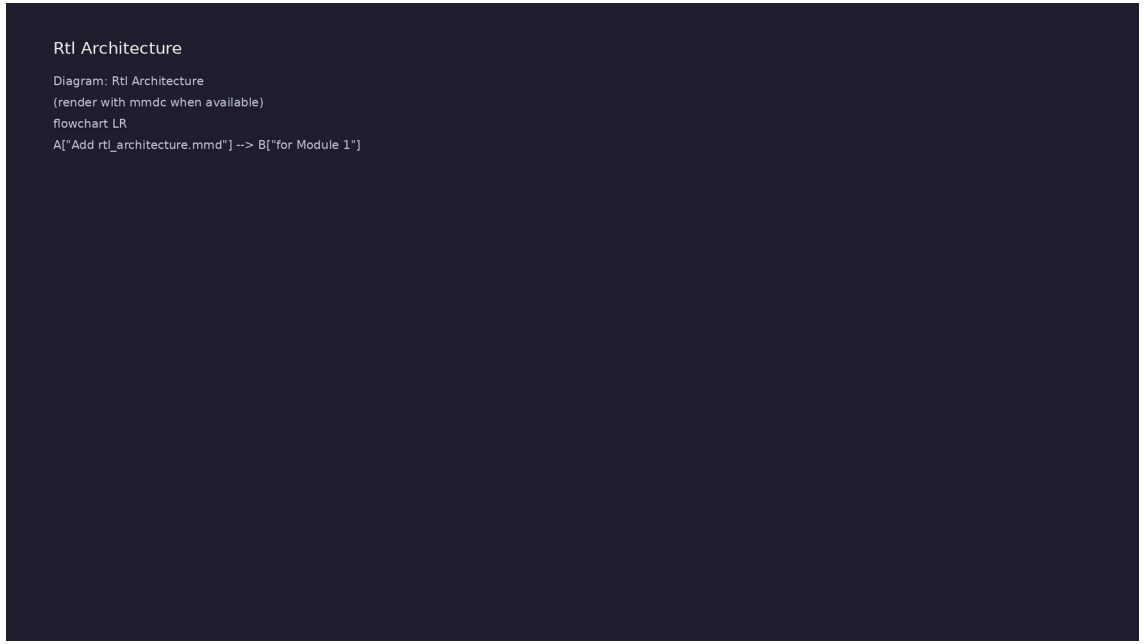
Overview

- This module establishes the foundation for verification work. You'll learn essential SystemVerilog...

Design architecture

1. Repository and example layout (1/3)

- Each example directory is self-contained with README and compile/run instructions
- DUTs are intentionally minimal so you focus on SystemVerilog language features
- UVM tests in `tests/uvm_tests/` preview patterns used from Module 3 onward
- module1/examples/sv_basics/: Classes, inheritance, polymorphism demos



1. Repository and example layout (2/3)

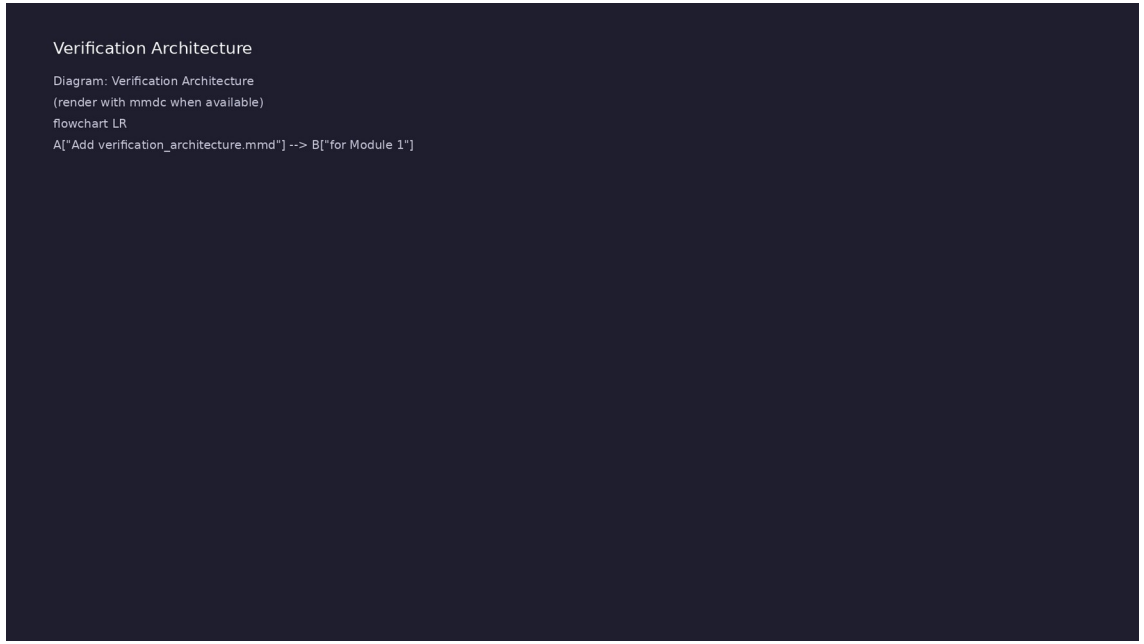
- module1/examples/interfaces/: Interfaces, modports, clocking blocks
- module1/examples/packages/: Packages and name visibility
- module1/dut/simple_gates/: Small RTL (AND gate, counter) for directed tests
- module1/tests/sv_tests/: Makefile-driven SV testbenches on simple DUTs

1. Repository and example layout (3/3)

- scripts/module1.sh: Orchestrates all examples and tests

2. Verification building blocks (1/2)

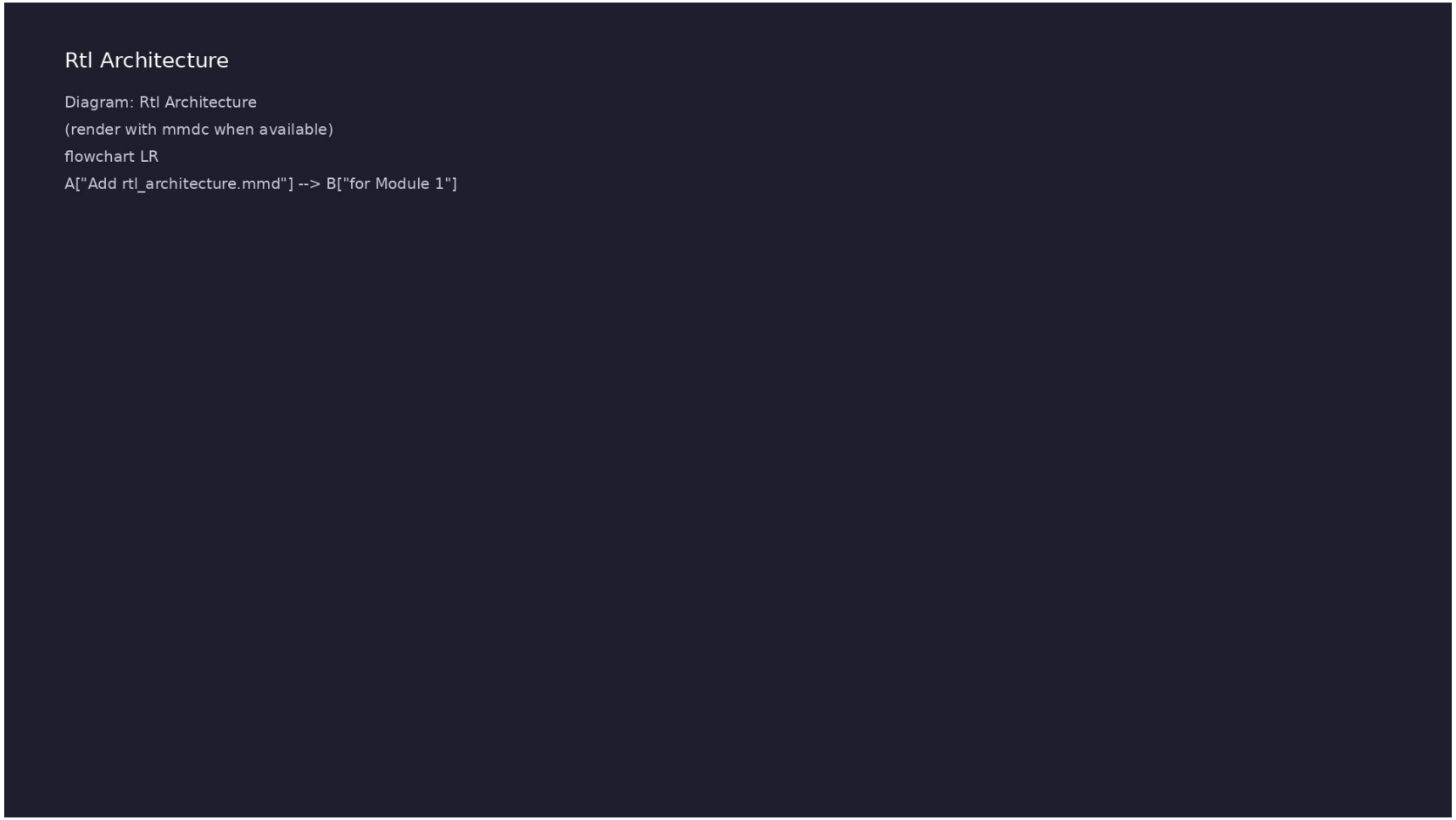
- Classes — transactions and models as reusable objects (`uvm_object` foundation)
- Interfaces — bundle DUT signals; connect testbench to RTL without port lists
- Packages — share types and parameters across testbench files
- Data structures — queues, associative arrays, dynamic arrays for stimulus/history



2. Verification building blocks (2/2)

- Layering: directed SV test → interface-based TB → optional UVM wrapper

RTL block diagram (reference)



Module 1: DUT hierarchy and signal flow.

Execution & simulation flow

How the example runs (toolchain)

- Makefile: Verilator + UVM_HOME compile RTL, interface, and test_*.sv
- UVM phases: build → connect → run_test → report (same pattern as Module 2)
- Sequence → driver → DUT → monitor → scoreboard (read SCOREBOARD at end)
- Typical command: make SIM=verilator TEST=test_* (see module1 demo slide)
- Loopback / hooks connect monitor observations to scoreboard checks

Example directory layout (1)

- module1/examples/sv_basics/: Classes, inheritance, polymorphism demos
- module1/examples/interfaces/: Interfaces, modports, clocking blocks
- module1/examples/packages/: Packages and name visibility
- module1/dut/simple_gates/: Small RTL (AND gate, counter) for directed tests
- module1/tests/sv_tests/: Makefile-driven SV testbenches on simple DUTs

Example directory layout (2)

- scripts/module1.sh: Orchestrates all examples and tests

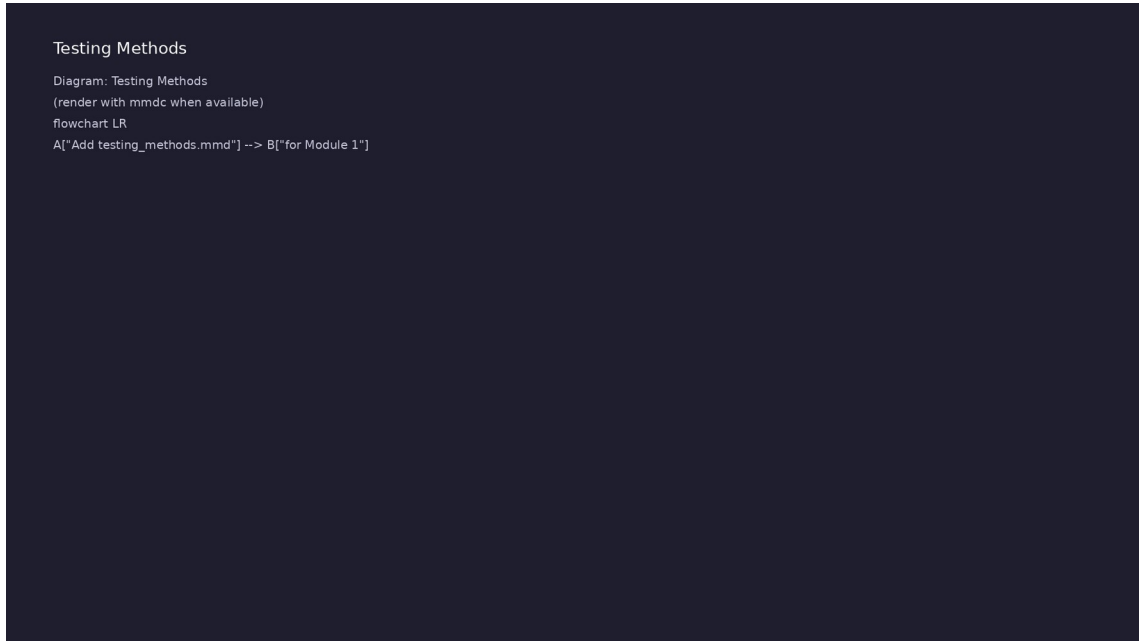
Directed test execution sequence

- make SIM=verilator TEST=test_and_gate
- compile
- run
- read result

Verification & testing methods

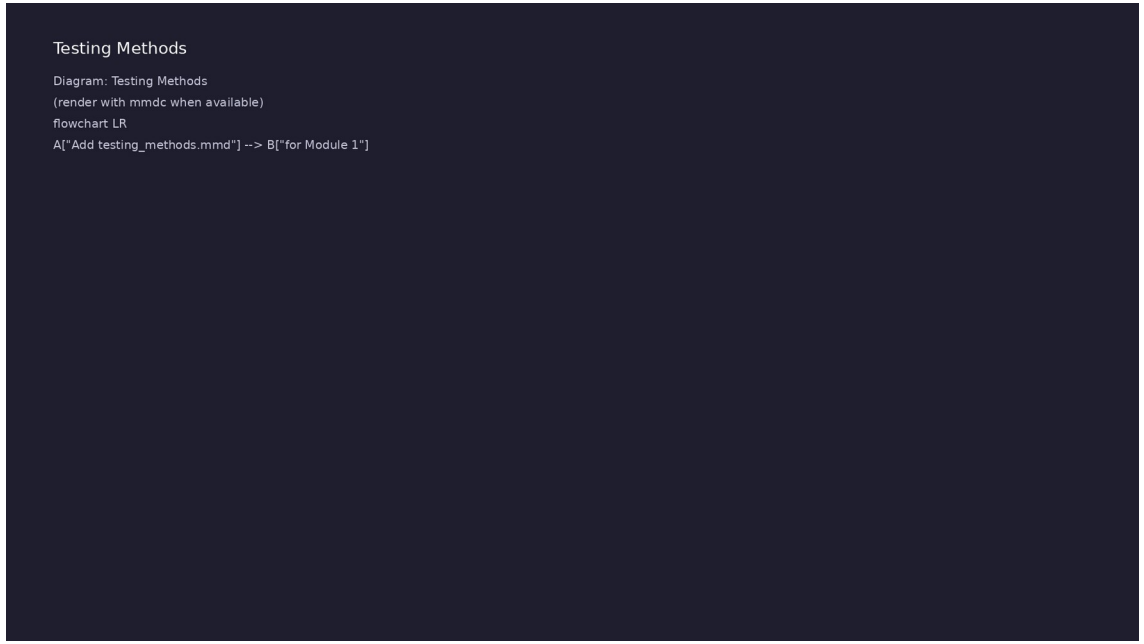
1. Directed self-check methodology

- Instantiate DUT, apply known stimulus in `initial` block or C++ main
- Compare outputs in-test; print `PASS` / `FAIL` (no external scoreboard yet)
- Use `$display` / `$error` for visibility; graduate to structured logging in Module 2
- Simulation flow: `make SIM=verilator TEST=test_and_gate` → compile → run → read result



2. Baseline test closure (1/2)

- Stimulus: fixed vectors covering corner cases (0, 1, overflow, reset)
- Checking: inline assertions and explicit comparisons against expected values
- Coverage: manual review of cases exercised (functional coverage in Module 5)
- Regression: `./scripts/module1.sh --all-sv` runs all example compilations



2. Baseline test closure (2/2)

- Out of scope: constrained-random, scoreboards, multi-agent coordination (later modules)

Syllabus topics

Protocol & design details

Detail: Example 1.1: Transaction Classes (`module1/examples/sv_basics...

- Base `Transaction` class with static variables (`id_counter`)
- Instance variables (`id`, `data`, `timestamp`)
- Special methods: `new()`, `copy()`, `compare()`, `convert2string()`
- Inheritance: `ReadTransaction` and `WriteTransaction` inherit from `Transaction`

Detail: Example 1.1: Transaction Classes (`module1/examples/sv_basics...

- Method overriding: Child classes override `convert2string()` method
- Using `super` to call parent class methods
- Purpose: Initialize object when created
- Called: Automatically via `Transaction txn = new();`

Detail: Example 1.2: Interfaces and Modports (`module1/examples/inter...

- Interface Definition: `apb_if` interface with signals and clocking block
- Modports: `master_mp` and `slave_mp` modports with different signal directions
- Clocking Block: `cb` clocking block for synchronous access
- Interface Usage: Module using interface with modport

Detail: Example 1.2: Interfaces and Modports (`module1/examples/inter…

- Testbench Integration: Testbench connecting to interface
- Task Usage: Clock generation task, APB transaction tasks
- Purpose: Generate free-running clock signal
- Built-in Features: `forever` loop, delay operator `#`, bitwise NOT `~`

Detail: Example 1.3: Packages (`module1/examples/packages/package_exa...

- Package Definition: `verification_pkg` with common types and functions
- Package Import: Using `import verification_pkg::*`
- Name Resolution: Handling name conflicts
- Package Organization: Structuring verification code

Detail: Example 1.3: Packages (`module1/examples/packages/package_exa...

- Function Definitions: `is_valid_address()`, `calculate_checksum()` functions
- Task Definitions: `wait_for_condition()` task with timeout
- Purpose: Validates address range (returns 1 if valid, 0 if invalid)
- Usage: `if (is_valid_address(addr)) ...`

Detail: Example 1.4: Data Structures (`module1/examples/data_structur...

- Dynamic Arrays: Transaction arrays that grow dynamically
- Queues: FIFO queues for transaction management
- Associative Arrays: Indexed by transaction ID
- Structures: Complex data structures for verification

Detail: Example 1.4: Data Structures (`module1/examples/data_structur…

- Array Methods: Using built-in array methods
- Class Methods: Queue and scoreboard methods (`push()`, `pop()`, `check()`)
- `push()`: Wraps `push_back()` for FIFO enqueue
- `pop()`: Wraps `pop_front()` for FIFO dequeue

Detail: Example 1.5: Error Handling (`module1/examples/error_handling...

- UVM Reporting: Using `uvm_report_info()`, `uvm_report_error()`, etc.
- Assertions: Immediate and concurrent assertions
- Error Handling: Try-catch blocks for exception handling
- Logging Patterns: Structured logging for verification

Detail: Example 1.5: Error Handling (`module1/examples/error_handling...

- Retry Logic: Task with retry mechanism using `retry_task()`
- Exception Objects: Custom exception class with `convert2string()` method
- Constructor: Initializes exception with message and code
- `convert2string()`: Formats exception for display

Detail: Functions vs Tasks (1)

- Purpose: Compute and return a single value
- Return Type: Must declare a return type (`void`, `int`, `string`, `bit`, etc.)
- Timing: Execute in zero simulation time (no delays allowed)
- Usage: For calculations, data transformations, comparisons

Detail: Functions vs Tasks (2)

- Syntax: `function <return_type> <name>(<arguments>);
  ... endfunction`
- When to Use:
- □ Data validation (`is_valid_address()`)
- □ Calculations (`calculate_checksum()`)

Detail: Built-in SystemVerilog Functions (1)

- Purpose: Print formatted text to console (like
  `printf` in C)
- Format Specifiers:
- `%0d` - Decimal integer (no leading zeros)
- `%0h` or `%0x` - Hexadecimal (no leading zeros)

Detail: Built-in SystemVerilog Functions (2)

- `%0b` - Binary
- `%0s` - String
- `%0t` - Time
- `%0f` - Real (floating point)

Detail: Class Methods (Functions in Classes) (1)

- Purpose: Initialize object when created
- Called: Automatically when using `new()` operator
- Parameters: Can have default values
- Example: `Transaction txn = new();` creates object and calls `new()`

Detail: Class Methods (Functions in Classes) (2)

- Purpose: Create deep copy of object
- Usage: Duplicate transactions for scoreboards, queues
- Null Check: Always check for null before accessing object fields
- Example: `txn_copy.copy(txn1);` copies all fields from `txn1` to `txn_copy`

Detail: Tasks in Testbenches (1)

- Purpose: Organize test code into reusable units
- Timing: Can contain delays (`#10`) and event waits (`@(posedge clk)`)
- Usage: Each test case is a separate task
- Calling: `test_basic();` executes the task

Detail: Tasks in Testbenches (2)

- Purpose: Encapsulate common sequences (reset, initialization)
- Reusability: Can be called from multiple test tasks
- Usage: `reset();` applies reset sequence before each test
- Purpose: Pass data to tasks

Detail: Array Methods (1)

- `push_back()`: Add element to end (FIFO enqueue)
- `pop_front()`: Remove element from front (FIFO dequeue)
- `size()`: Return number of elements
- Usage: Transaction queues, scoreboards

Detail: Array Methods (2)

- `exists()`: Check if key exists in associative array
- Indexing: Use any integral type as key
- Usage: Scoreboards, coverage collectors

Detail: Function and Task Best Practices (1)

- Functions execute in zero time
- Perfect for data transformations, comparisons
- Must return a value (or `void`)
- Tasks can contain delays

Detail: Function and Task Best Practices (2)

- Perfect for test sequences, stimulus generation
- No return value
- Always check for null before accessing object fields
- Prevents simulation crashes

Detail: Real-World Testbench Examples (1)

- Uses delays (`#10`) for signal propagation
- Uses `assert` with `$error()` for verification
- Uses `$display()` for test output
- Organized as separate task for reusability

Detail: Real-World Testbench Examples (2)

- Helper task (`reset()`) encapsulates common sequence
- Uses `@(posedge clk)` for clock synchronization
- Combines delays and event control
- Uses type casting (`8'(i)`) for bit width matching

Detail: Performance Considerations (1)

- Functions: Execute in zero time, no simulation overhead
- Tasks: Consume simulation time, have scheduling overhead
- Recommendation: Use functions for calculations, tasks only when timing is needed
- `push_back()` / `pop_front()`: O(1) for queues

Detail: Performance Considerations (2)

- `size()`: O(1) for queues and arrays
- `exists()`: O(1) average for associative arrays
- `foreach`: O(n) where n is number of elements

Detail: Troubleshooting Guide (1)

- Cause: Trying to use `#`, `@`, or `wait` in a function
- Solution: Convert to task or remove timing control
- Cause: Accessing object fields without null check
- Solution: Always check `if (obj == null)` before accessing

Detail: Troubleshooting Guide (2)

- Cause: Trying to use task return value: `int x = task_name();`
- Solution: Use output parameter: `task_name(output int x);`
- Cause: Function declared with return type but no return statement
- Solution: Add `return` statement or change to `void` function

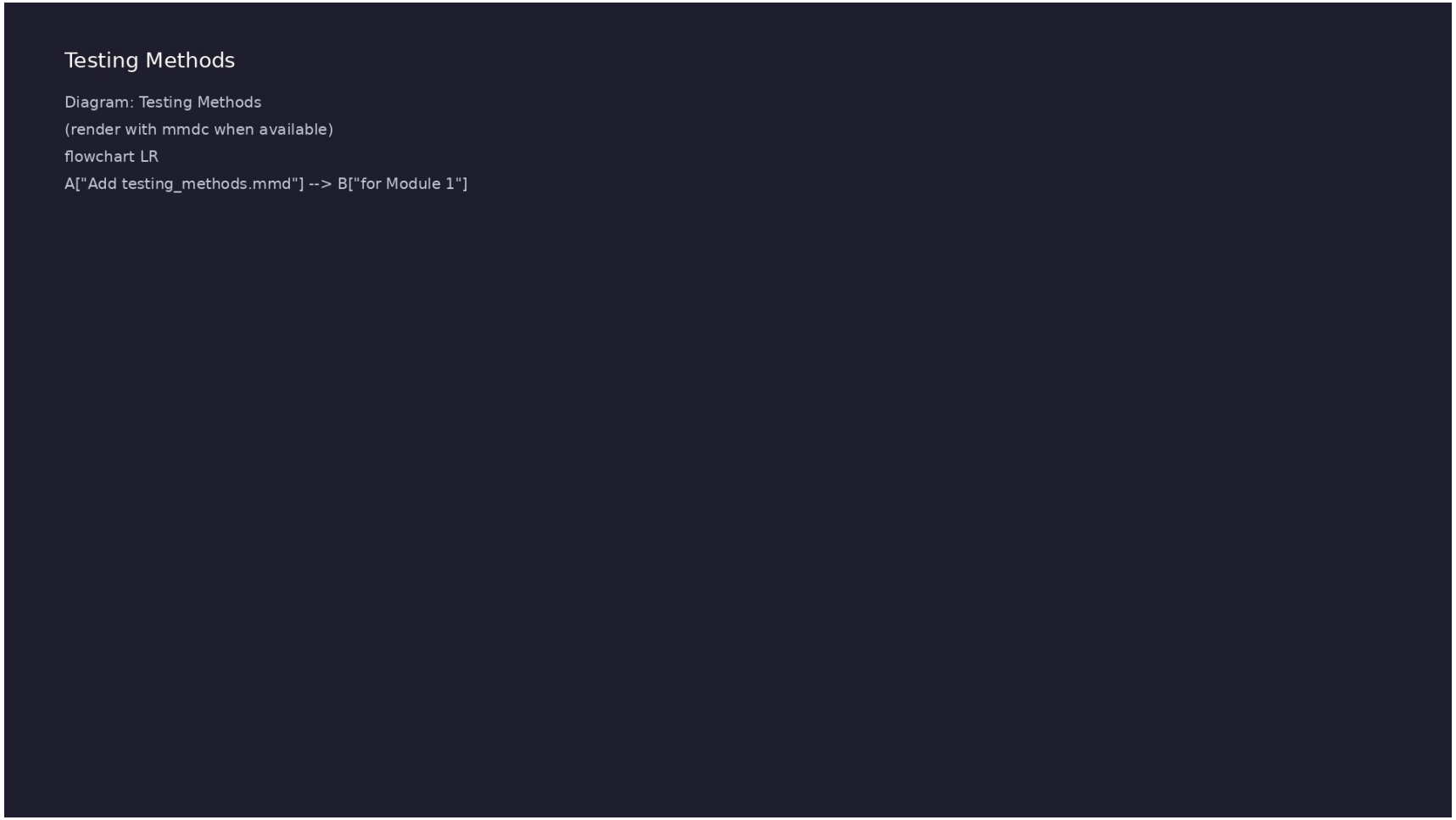
Detail: Summary (1)

- Functions: Zero-time operations that return values; use for calculations
- Tasks: Time-consuming operations; use for test sequences
- Class Methods: Functions within classes; enable object-oriented programming
- Built-in Functions: `$display()`, `$sformatf()`, `$finish()`, `$error()`

Detail: Summary (2)

- Array Methods: `push_back()`, `pop_front()`, `size()`, `exists()`
- Best Practices: Null checks, method overriding, task organization
- Common Pitfalls: Delays in functions, missing null checks, wrong assignment types
- Performance: Functions are faster; use tasks only when timing is needed

Verification flow (reference)



Stimulus, DUT response, and check points for this module.

1. SystemVerilog Classes and Inheritance for Verification (1/9)

- Object-Oriented Programming Basics
- Classes and objects
- Instance variables and methods
- Static variables and methods
- Encapsulation concepts
- Access modifiers (public, protected, local)

1. SystemVerilog Classes and Inheritance for Verification (2/9)

- Inheritance in SystemVerilog
- Single inheritance
- Method overriding
- `super` keyword usage
- Virtual methods and polymorphism
- Abstract classes

1. SystemVerilog Classes and Inheritance for Verification (3/9)

- Special Methods
- `new()` constructor
- `copy()` and `clone()` methods
- `compare()` and `convert2string()` methods
- `print()` and `sprint()` methods
- Class Design Patterns for Verification

1. SystemVerilog Classes and Inheritance for Verification (4/9)

- Factory pattern basics
- Singleton pattern
- Builder pattern
- Strategy pattern
- Base `Transaction` class with static variables (`id_counter`)
- Instance variables (`id`, `data`, `timestamp`)

1. SystemVerilog Classes and Inheritance for Verification (5/9)

- Special methods: `new()`, `copy()`, `compare()`, `convert2string()`
- Inheritance: `ReadTransaction` and `WriteTransaction` inherit from `Transaction`
- Method overriding: Child classes override `convert2string()` method
- Using `super` to call parent class methods
- Purpose: Initialize object when created
- Called: Automatically via `Transaction txn = new();`

1. SystemVerilog Classes and Inheritance for Verification (6/9)

- Default Parameters: `name` parameter has default value
- Purpose: Deep copy of transaction object
- Null Check: Always validates input before accessing
- Usage: `txn_copy.copy(txn1);` copies all fields
- Returns: `1` if equal, `0` if not equal
- Usage: `if (txn1.compare(txn2)) $display("Match!");`

1. SystemVerilog Classes and Inheritance for Verification (7/9)

- Uses: `$sformatf()` built-in function for formatting
- Format Specifiers: `%0d` (decimal), `%0h` (hexadecimal)
- Usage: `$display("%s", txn.convert2string());`
- Purpose: Encapsulated access to private fields
- Usage: `int id_val = txn.get_id();`
- Polymorphism: Derived class method called when using base class handle

1. SystemVerilog Classes and Inheritance for Verification (8/9)

- Base transaction creation and manipulation
- Derived transaction examples showing inheritance
- Copy and compare operations working correctly
- String conversion examples
- Static Variables: `id_counter` is shared across all instances
- Instance Variables: Each transaction has its own `id`, `data`, `timestamp`

1. SystemVerilog Classes and Inheritance for Verification (9/9)

- Special Methods: Enable UVM-compatible behavior (copy, compare, string conversion)
- Inheritance: `ReadTransaction` and `WriteTransaction` extend base functionality
- Method Overriding: Child classes customize string representation
- Function Syntax: `function <return_type> <name>(<args>); ... endfunction`
- Null Safety: Always check for null before accessing object fields
- Built-in Functions: `$sformatf()`, `$error()`, `$display()` used throughout

2. Interfaces and Modports (1/8)

- SystemVerilog Interfaces
- Interface definition
- Interface signals
- Interface methods
- Interface instantiation
- Modports

2. Interfaces and Modports (2/8)

- Modport definition
- Direction specification (input, output, inout)
- Modport usage in modules
- Multiple modports per interface
- Clocking Blocks
- Clocking block definition

2. Interfaces and Modports (3/8)

- Clocking block signals
- Synchronous signal access
- Clocking block in interfaces
- Verification-Specific Interfaces
- Driver interfaces
- Monitor interfaces

2. Interfaces and Modports (4/8)

- Scoreboard interfaces
- Agent interfaces
- Interface Definition: `apb_if` interface with signals and clocking block
- Modports: `master_mp` and `slave_mp` modports with different signal directions
- Clocking Block: `cb` clocking block for synchronous access
- Interface Usage: Module using interface with modport

2. Interfaces and Modports (5/8)

- Testbench Integration: Testbench connecting to interface
- Task Usage: Clock generation task, APB transaction tasks
- Purpose: Generate free-running clock signal
- Built-in Features: `forever` loop, delay operator `#`, bitwise NOT `~`
- Result: Clock with 10 time unit period (50% duty cycle)
- Event Control: `@(posedge clk)` waits for clock edge

2. Interfaces and Modports (6/8)

- Non-blocking Assignment: `<=` for sequential logic
- Wait Loop: `while (!condition) @(posedge clk)` pattern
- Blocking Assignment: `=` for initialization (combinational)
- Non-blocking Assignment: `<=` for sequential logic (clocked)
- Interface Signal Access: `bus.signal_name` accesses interface signals
- Interfaces provide structured communication between modules

2. Interfaces and Modports (7/8)

- Modports control signal direction and access
- Clocking blocks enable synchronous signal access
- Interfaces are essential for UVM testbenches
- Event Control: `@(posedge clk)` synchronizes to clock edges
- Assignment Types: `=` (blocking) vs `<=` (non-blocking)
- Wait Patterns: `while (!condition) @(posedge clk)` for handshaking

2. Interfaces and Modports (8/8)

- Built-in Operators: `~` (NOT), `&&` (AND), `!` (logical NOT)

3. Packages and Namespaces (1/7)

- SystemVerilog Packages
- Package definition
- Package items (classes, functions, types)
- Package import
- Package compilation order
- Namespaces

3. Packages and Namespaces (2/7)

- Package namespaces
- Resolving name conflicts
- Wildcard imports
- Explicit imports
- Verification Packages
- Common verification types

3. Packages and Namespaces (3/7)

- Utility functions
- Shared classes
- Configuration types
- Package Definition: `verification_pkg` with common types and functions
- Package Import: Using `import verification_pkg::*`
- Name Resolution: Handling name conflicts

3. Packages and Namespaces (4/7)

- Package Organization: Structuring verification code
- Function Definitions: `is_valid_address()`, `calculate_checksum()` functions
- Task Definitions: `wait_for_condition()` task with timeout
- Purpose: Validates address range (returns 1 if valid, 0 if invalid)
- Usage: `if (is_valid_address(addr)) ...`
- Key Feature: `automatic` keyword makes function re-entrant

3. Packages and Namespaces (5/7)

- Purpose: Calculates XOR checksum of 32-bit data
- Returns: 8-bit checksum value
- Usage: `logic [7:0] checksum =
  calculate_checksum(data);`
- Purpose: Wait for condition with timeout
- Key Features:
- `ref` parameter passes by reference

3. Packages and Namespaces (6/7)

- Contains delays (`#1`)
- Demonstrates task with timing control
- Packages organize related code (types, functions, tasks, constants)
- Import statements control namespace
- Functions in packages are reusable across modules
- `automatic` keyword enables re-entrant functions/tasks

3. Packages and Namespaces (7/7)

- Tasks can contain delays, functions cannot
- Packages improve code reusability
- Essential for UVM library organization

4. Data Structures for Verification (1/8)

- SystemVerilog Data Types
- `bit` and `logic` types
- `int`, `longint`, `shortint`
- `real` and `realtime`
- `string` type
- `enum` types

4. Data Structures for Verification (2/8)

- Arrays
- Fixed-size arrays
- Dynamic arrays
- Associative arrays
- Queues
- Array methods

4. Data Structures for Verification (3/8)

- Structures and Unions
- `struct` definition
- Packed and unpacked structures
- `union` types
- Structure methods
- Verification-Specific Data Structures

4. Data Structures for Verification (4/8)

- Transaction queues
- Coverage bins
- Scoreboard structures
- Configuration structures
- Dynamic Arrays: Transaction arrays that grow dynamically
- Queues: FIFO queues for transaction management

4. Data Structures for Verification (5/8)

- Associative Arrays: Indexed by transaction ID
- Structures: Complex data structures for verification
- Array Methods: Using built-in array methods
- Class Methods: Queue and scoreboard methods
  (`push()`, `pop()`, `check()`)
- `push()`: Wraps `push_back()` for FIFO enqueue
- `pop()`: Wraps `pop_front()` for FIFO dequeue

4. Data Structures for Verification (6/8)

- `size()`: Returns queue element count
- Usage: `queue.push(txn); txn = queue.pop();`
- `add_expected()`: Stores expected values
- `check()`: Compares expected vs actual using `exists()` method
- Usage: `scoreboard.add_expected(1, data); scoreboard.check(1);`
- `sample_address()`: Records address access

4. Data Structures for Verification (7/8)

- `get_coverage()`: Calculates coverage percentage
- Built-in Features: `foreach` loop, `exists()` method, type casting
- Dynamic arrays provide flexibility
- Queues are ideal for FIFO operations
- Associative arrays enable key-value lookups
- Structures organize related data

4. Data Structures for Verification (8/8)

- Array methods (`push_back()`, `pop_front()`, `size()`, `exists()`) are essential
- Wrapper methods provide clean interfaces to built-in array operations
- `foreach` loops iterate over associative arrays efficiently

5. Error Handling and Logging (1/7)

- SystemVerilog Assertions
- Immediate assertions
- Concurrent assertions
- Assertion severity
- Assertion coverage
- UVM Reporting

5. Error Handling and Logging (2/7)

- UVM messaging system
- Severity levels (FATAL, ERROR, WARNING, INFO, DEBUG)
- Verbosity levels
- Message formatting
- Exception Handling
- Try-catch blocks (SystemVerilog 2017+)

5. Error Handling and Logging (3/7)

- Error propagation
- Graceful error handling
- Logging Patterns
- Structured logging
- Log file management
- Debug logging

5. Error Handling and Logging (4/7)

- Production logging
- UVM Reporting: Using `uvm_report_info()`, `uvm_report_error()`, etc.
- Assertions: Immediate and concurrent assertions
- Error Handling: Try-catch blocks for exception handling
- Logging Patterns: Structured logging for verification
- Retry Logic: Task with retry mechanism using `retry_task()`

5. Error Handling and Logging (5/7)

- Exception Objects: Custom exception class with `convert2string()` method
- Constructor: Initializes exception with message and code
- `convert2string()`: Formats exception for display
- Usage: `$display("%s", exc.convert2string());`
- Purpose: Retry operation with configurable attempts
- Built-in Functions Used:

5. Error Handling and Logging (6/7)

- `$display()`: Print formatted messages
- `$urandom_range()`: Generate random numbers for simulation
- Usage: `comp.retry_task(3);` retries up to 3 times
- Purpose: Demonstrate different message severity levels
- Usage: Shows structured logging patterns
- UVM reporting provides standardized messaging

5. Error Handling and Logging (7/7)

- Assertions catch design errors early
- Exception handling enables graceful failures
- Logging is essential for debugging
- Tasks can implement retry logic with delays
- Built-in functions like `$urandom_range()` are useful for simulation
- Custom exception classes provide structured error information

6. SystemVerilog Functions and Tasks (1/26)

- Purpose: Compute and return a single value
- Return Type: Must declare a return type (`void`, `int`, `string`, `bit`, etc.)
- Timing: Execute in zero simulation time (no delays allowed)
- Usage: For calculations, data transformations, comparisons
- Syntax: `function <return_type> <name>(<arguments>); ... endfunction`
- When to Use:

6. SystemVerilog Functions and Tasks (2/26)

- ✓ Data validation (`is_valid_address()`)
- ✓ Calculations (`calculate_checksum()`)
- ✓ Comparisons (`compare()`)
- ✓ String formatting (`convert2string()`)
- ✓ Accessor methods (`get_id()`)
- × Operations requiring delays

6. SystemVerilog Functions and Tasks (3/26)

- × Test sequences
- Purpose: Perform operations that may consume time
- Return Type: No return value (implicitly `void`)
- Timing: Can contain delays (`#`, `@`, `wait`)
- Usage: For test sequences, stimulus generation, complex operations
- Syntax: `task <name>(<arguments>); ... endtask`

6. SystemVerilog Functions and Tasks (4/26)

- When to Use:
- □ Test sequences (`test_basic()`, `test_reset()`)
- □ Reset/initialization sequences (`reset()`)
- □ Clock generation
- □ Protocol transactions (APB write/read)
- □ Retry logic with delays

6. SystemVerilog Functions and Tasks (5/26)

- × Simple calculations (use function instead)
- × Data transformations (use function instead)
- Purpose: Print formatted text to console (like `printf` in C)
- Format Specifiers:
- `%0d` - Decimal integer (no leading zeros)
- `%0h` or `%0x` - Hexadecimal (no leading zeros)

6. SystemVerilog Functions and Tasks (6/26)

- `%0b` - Binary
- `%0s` - String
- `%0t` - Time
- `%0f` - Real (floating point)
- `%0e` - Scientific notation
- Variants:

6. SystemVerilog Functions and Tasks (7/26)

- `$display()` - Standard output
- `$displayb()` - Binary format
- `$displayh()` - Hexadecimal format
- `$displayo()` - Octal format
- Example from code: Used extensively in all examples
  for test output
- Real Example (from `test_counter.sv`):

6. SystemVerilog Functions and Tasks (8/26)

- Purpose: Format string with placeholders (like `sprintf` in C)
- Returns: Formatted string (doesn't print to console)
- Usage: Create string representations of objects
- Example from code: Used in `convert2string()` methods to format transaction data
- Real Example (from `transaction.sv`):
- Purpose: Stop simulation and exit

6. SystemVerilog Functions and Tasks (9/26)

- Usage: End testbench execution after tests complete
- Exit Codes: 0 = success, 1 = error, 2 = fatal
- Example from code: Called at end of all test programs
- Real Example (from `test_and_gate.sv`):
- Purpose: Report errors with formatted messages
- Usage: In assertions and error handling

6. SystemVerilog Functions and Tasks (10/26)

- Severity: ERROR level (stops simulation if severity is high enough)
- Example from code: Used in test assertions when checks fail
- Real Example (from `test_and_gate.sv`):
- Purpose: Different severity levels for reporting
- Usage: Use appropriate level for the situation
- Severity Order: FATAL > ERROR > WARNING > INFO

6. SystemVerilog Functions and Tasks (11/26)

- Purpose: Get current simulation time
- Usage: Timing verification, logging with timestamps
- Purpose: Generate random numbers for test stimulus
- Usage: Random test generation, constrained random verification
- Example from code: Used in `error_handling_example.sv` for retry simulation
- Purpose: Safe type casting between compatible types

6. SystemVerilog Functions and Tasks (12/26)

- Returns: `1` if cast succeeds, `0` if fails
- Usage: Polymorphic object handling, type checking
- Purpose: Initialize object when created
- Called: Automatically when using `new()` operator
- Parameters: Can have default values
- Example: `Transaction txn = new();` creates object
  and calls `new()`

6. SystemVerilog Functions and Tasks (13/26)

- Purpose: Create deep copy of object
- Usage: Duplicate transactions for scoreboards, queues
- Null Check: Always check for null before accessing object fields
- Example: `txn_copy.copy(txn1);` copies all fields from `txn1` to `txn_copy`
- Purpose: Compare two objects for equality
- Returns: `1` if equal, `0` if not equal

6. SystemVerilog Functions and Tasks (14/26)

- Usage: Scoreboard checking, test verification
- Example: `if (txn1.compare(txn2))
  $display("Match!");`
- Purpose: Create human-readable string representation
- Returns: Formatted string with object fields
- Usage: Debugging, logging, test output
- Example: `$display("%s", txn.convert2string());`

6. SystemVerilog Functions and Tasks (15/26)

- Purpose: Provide controlled access to private fields
- Usage: Encapsulation, data access
- Example: `int id_val = txn.get_id();`
- Purpose: Organize test code into reusable units
- Timing: Can contain delays (`#10`) and event waits (`@(posedge clk)`)
- Usage: Each test case is a separate task

6. SystemVerilog Functions and Tasks (16/26)

- Calling: `test_basic();` executes the task
- Purpose: Encapsulate common sequences (reset, initialization)
- Reusability: Can be called from multiple test tasks
- Usage: `reset();` applies reset sequence before each test
- Purpose: Pass data to tasks
- Usage: Make tasks flexible and reusable

6. SystemVerilog Functions and Tasks (17/26)

- Calling: `write_transaction(16'h1000, 32'hDEADBEEF);`
- `push_back()`: Add element to end (FIFO enqueue)
- `pop_front()`: Remove element from front (FIFO dequeue)
- `size()`: Return number of elements
- Usage: Transaction queues, scoreboards
- `exists()`: Check if key exists in associative array

6. SystemVerilog Functions and Tasks (18/26)

- Indexing: Use any integral type as key
- Usage: Scoreboards, coverage collectors
- Functions execute in zero time
- Perfect for data transformations, comparisons
- Must return a value (or `void`)
- Tasks can contain delays

6. SystemVerilog Functions and Tasks (19/26)

- Perfect for test sequences, stimulus generation
- No return value
- Always check for null before accessing object fields
- Prevents simulation crashes
- Derived classes can override base class methods
- Provides polymorphic behavior

6. SystemVerilog Functions and Tasks (20/26)

- Each test case as separate task
- Main `initial` block orchestrates test execution
- Improves code readability and maintainability
- Uses delays (`#10`) for signal propagation
- Uses `assert` with `$error()` for verification
- Uses `$display()` for test output

6. SystemVerilog Functions and Tasks (21/26)

- Organized as separate task for reusability
- Helper task (`reset()`) encapsulates common sequence
- Uses `@(posedge clk)` for clock synchronization
- Combines delays and event control
- Uses type casting (`8'(i)`) for bit width matching
- Uses nested loops for exhaustive testing

6. SystemVerilog Functions and Tasks (22/26)

- Type casting with `bit'(i)`
- Expression evaluation in assertion
- Multiple format specifiers in `$error()`
- Functions: Execute in zero time, no simulation overhead
- Tasks: Consume simulation time, have scheduling overhead
- Recommendation: Use functions for calculations, tasks only when timing is needed

6. SystemVerilog Functions and Tasks (23/26)

- `push_back()` / `pop_front()`: O(1) for queues
- `size()`: O(1) for queues and arrays
- `exists()`: O(1) average for associative arrays
- `foreach`: O(n) where n is number of elements
- Cause: Trying to use `#`, `@`, or `wait` in a function
- Solution: Convert to task or remove timing control

6. SystemVerilog Functions and Tasks (24/26)

- Cause: Accessing object fields without null check
- Solution: Always check `if (obj == null)` before accessing
- Cause: Trying to use task return value: `int x = task_name();`
- Solution: Use output parameter: `task_name(output int x);`
- Cause: Function declared with return type but no return statement
- Solution: Add `return` statement or change to `void` function

6. SystemVerilog Functions and Tasks (25/26)

- Cause: Using wrong format in `$display()` or `$sformatf()`
- Solution: Use `%0d` for decimal, `%0h` for hex, `%0b` for binary, `%0s` for string
- Functions: Zero-time operations that return values; use for calculations
- Tasks: Time-consuming operations; use for test sequences
- Class Methods: Functions within classes; enable object-oriented programming
- Built-in Functions: `$display()`, `$sformatf()`, `$finish()`, `$error()`

6. SystemVerilog Functions and Tasks (26/26)

- Array Methods: `push_back()`, `pop_front()`, `size()`, `exists()`
- Best Practices: Null checks, method overriding, task organization
- Common Pitfalls: Delays in functions, missing null checks, wrong assignment types
- Performance: Functions are faster; use tasks only when timing is needed

Run: Example 1.1: Transaction Classes

```
./scripts/module1.sh --sv-basics
cd module1/examples/sv_basics
verilator -sv --cc --exe transaction.sv -o transaction_example
make -C obj_dir -f Vtransaction_example.mk
./obj_dir/Vtransaction_example
```

Hands-on examples

Module 1 self-check

```
$ ./scripts/module1.sh --help
```

Terminal: module1_check

```
Usage: ./scripts/module1.sh [OPTIONS]

Module 1: SystemVerilog and Verification Basics
This script runs examples and tests for Module 1.

OPTIONS:
    SystemVerilog Examples:
        --sv-basics           Run SystemVerilog class basics examples
        --interfaces          Run interfaces and modports examples
        --packages            Run packages and namespaces examples
        --data-structures     Run data structures examples
        --error-handling      Run error handling and logging examples
        --all-sv              Run all SystemVerilog examples (default)
        --skip-sv             Skip all SystemVerilog examples

    Tests:
        --sv-tests            Run SystemVerilog testbenches
        --uvm-tests           Run UVM testbenches
        --all-tests           Run all tests

    Environment:
        --sim SIMULATOR       Simulator to use (default: verilator)
        --uvm-home DIR        UVM library directory (default: $UVM_HOME)
        --jobs N              Number of parallel build jobs (default: 8)
        --no-clean            Skip cleaning before build (faster rebuilds)

    Other:
        --help, -h            Show this help message

EXAMPLES:
    # Run all SystemVerilog examples (with parallel builds)
    ./scripts/module1.sh

    # Run only SystemVerilog basics
    ./scripts/module1.sh --sv-basics
```

Exercise scaffold

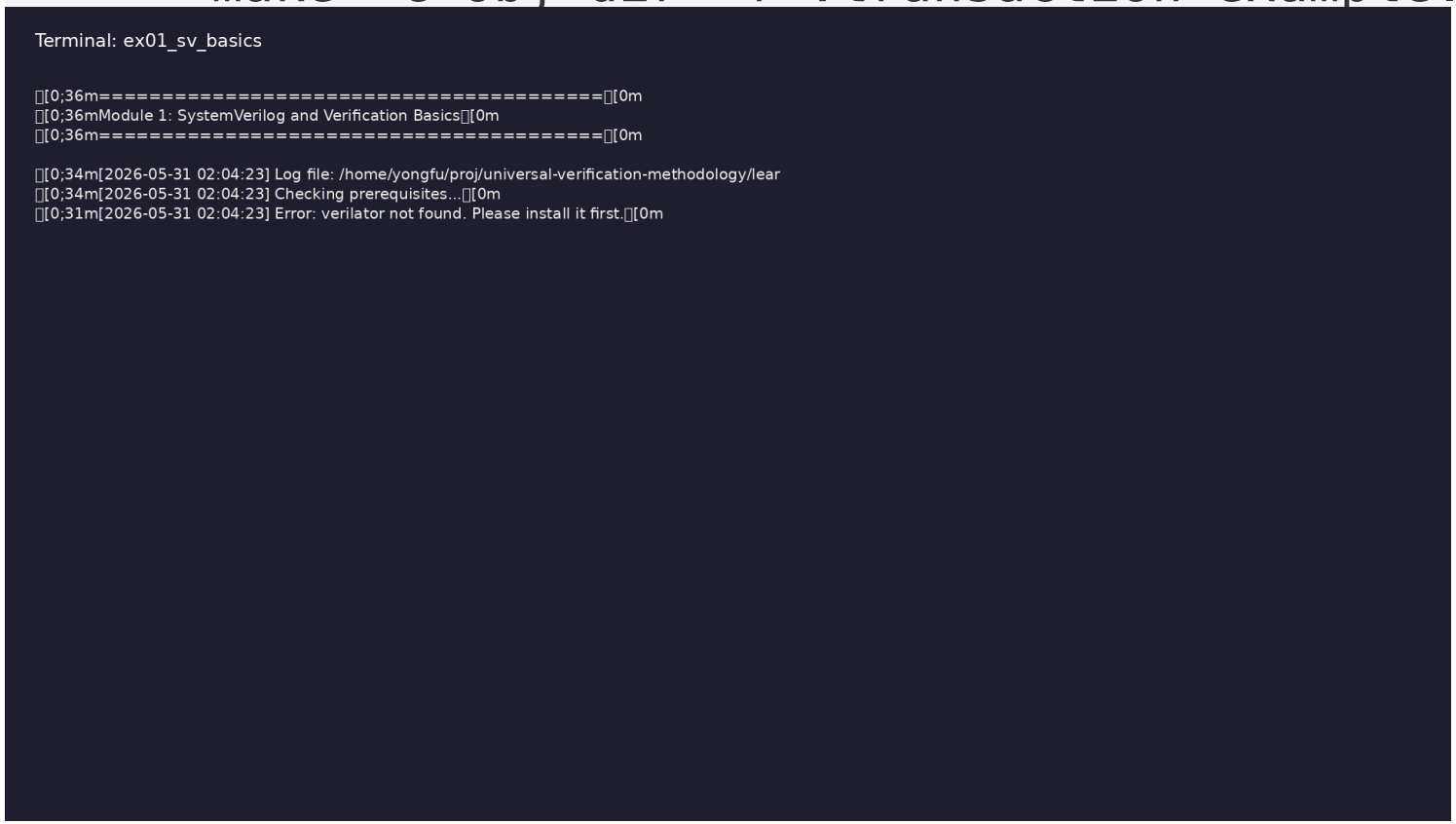
```
./scripts/module1.sh --scaffold
```

Example 1: Transaction Classes

- Base `Transaction` class with static variables (`id_counter`)
- Instance variables (`id`, `data`, `timestamp`)
- Special methods: `new()`, `copy()`, `compare()`, `convert2string()`
- Inheritance: `ReadTransaction` and `WriteTransaction` inherit from `Transaction`
- Method overriding: Child classes override `convert2string()` method

Demo: Transaction Classes

```
$ ./scripts/module1.sh --sv-basics && cd module1/examples/sv_basics &&
      verilator -sv --cc --exe transaction.sv -o transaction_example &&
      make -C obj_dir -f Vtransaction_example.mk
```

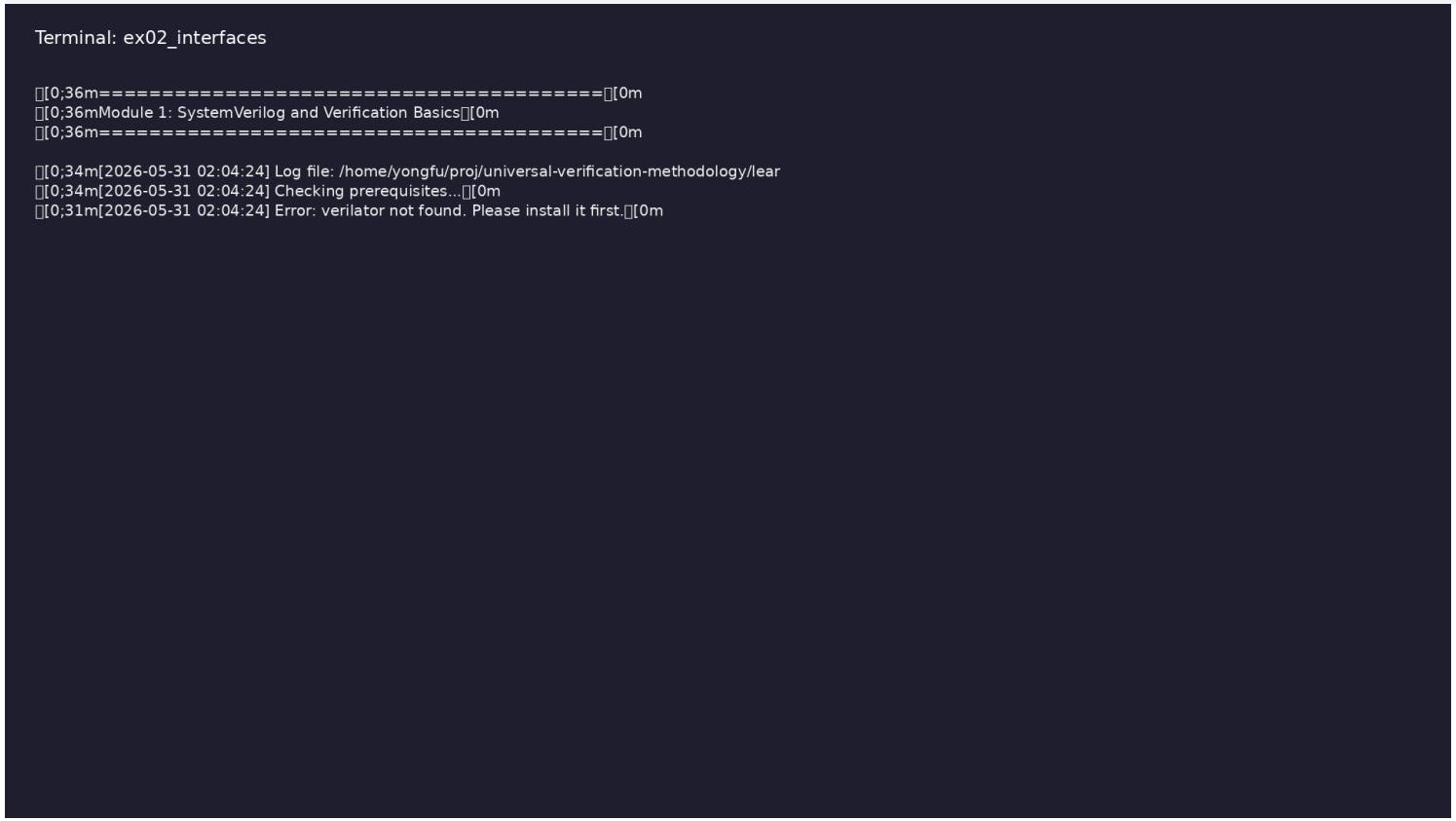


Example 2: Interfaces and Modports

- Interface Definition: `apb_if` interface with signals and clocking block
- Modports: `master_mp` and `slave_mp` modports with different signal directions
- Clocking Block: `cb` clocking block for synchronous access
- Interface Usage: Module using interface with modport
- Testbench Integration: Testbench connecting to interface

Demo: Interfaces and Modports

```
$ ./scripts/module1.sh --interfaces
```

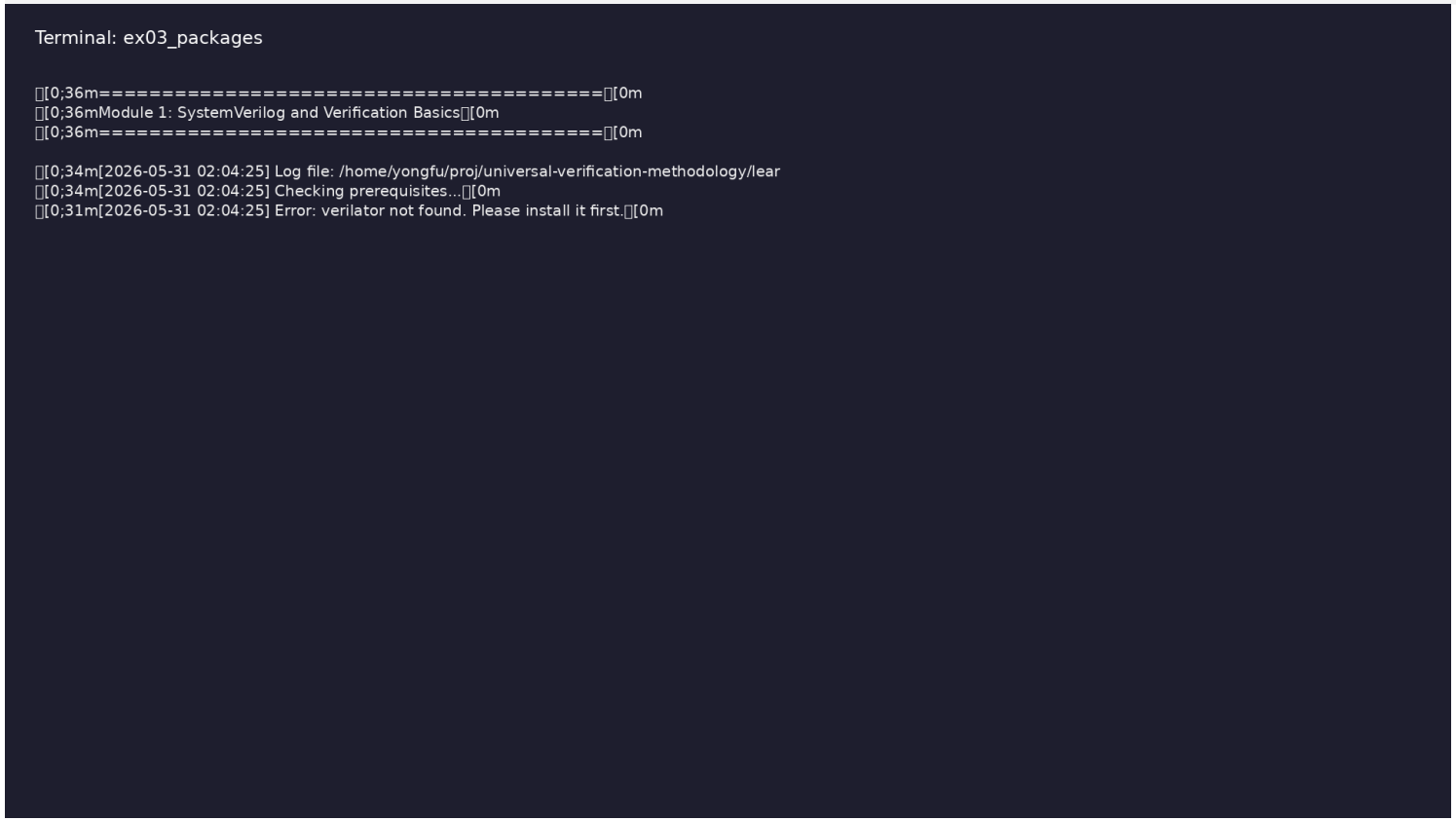


Example 3: Packages

- Package Definition: `verification_pkg` with common types and functions
- Package Import: Using `import verification_pkg::*`
- Name Resolution: Handling name conflicts
- Package Organization: Structuring verification code
- Function Definitions: `is_valid_address()`, `calculate_checksum()` functions

Demo: Packages

```
$ ./scripts/module1.sh --packages
```

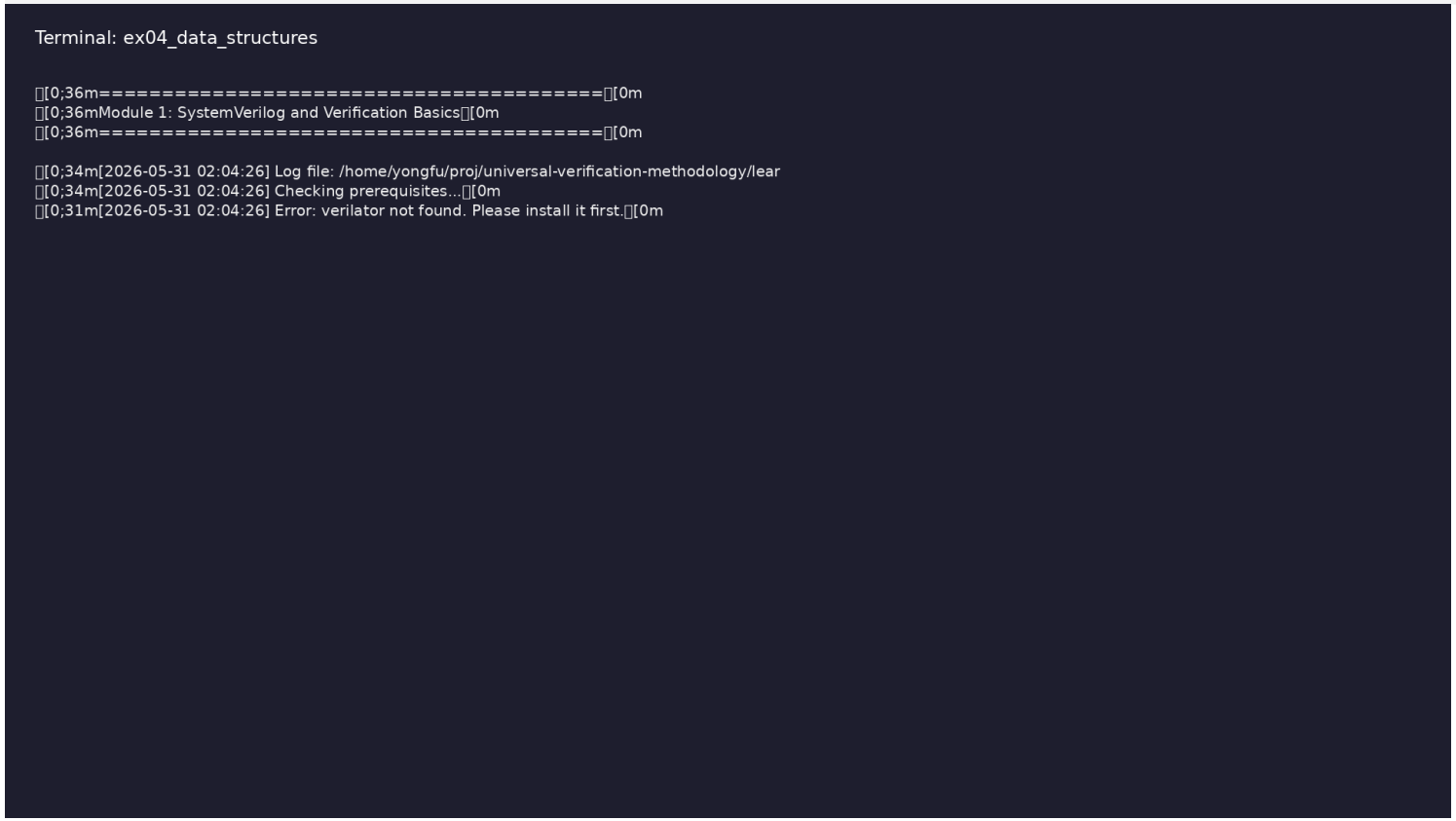


Example 4: Data Structures

- Dynamic Arrays: Transaction arrays that grow dynamically
- Queues: FIFO queues for transaction management
- Associative Arrays: Indexed by transaction ID
- Structures: Complex data structures for verification
- Array Methods: Using built-in array methods

Demo: Data Structures

```
$ ./scripts/module1.sh --data-structures
```

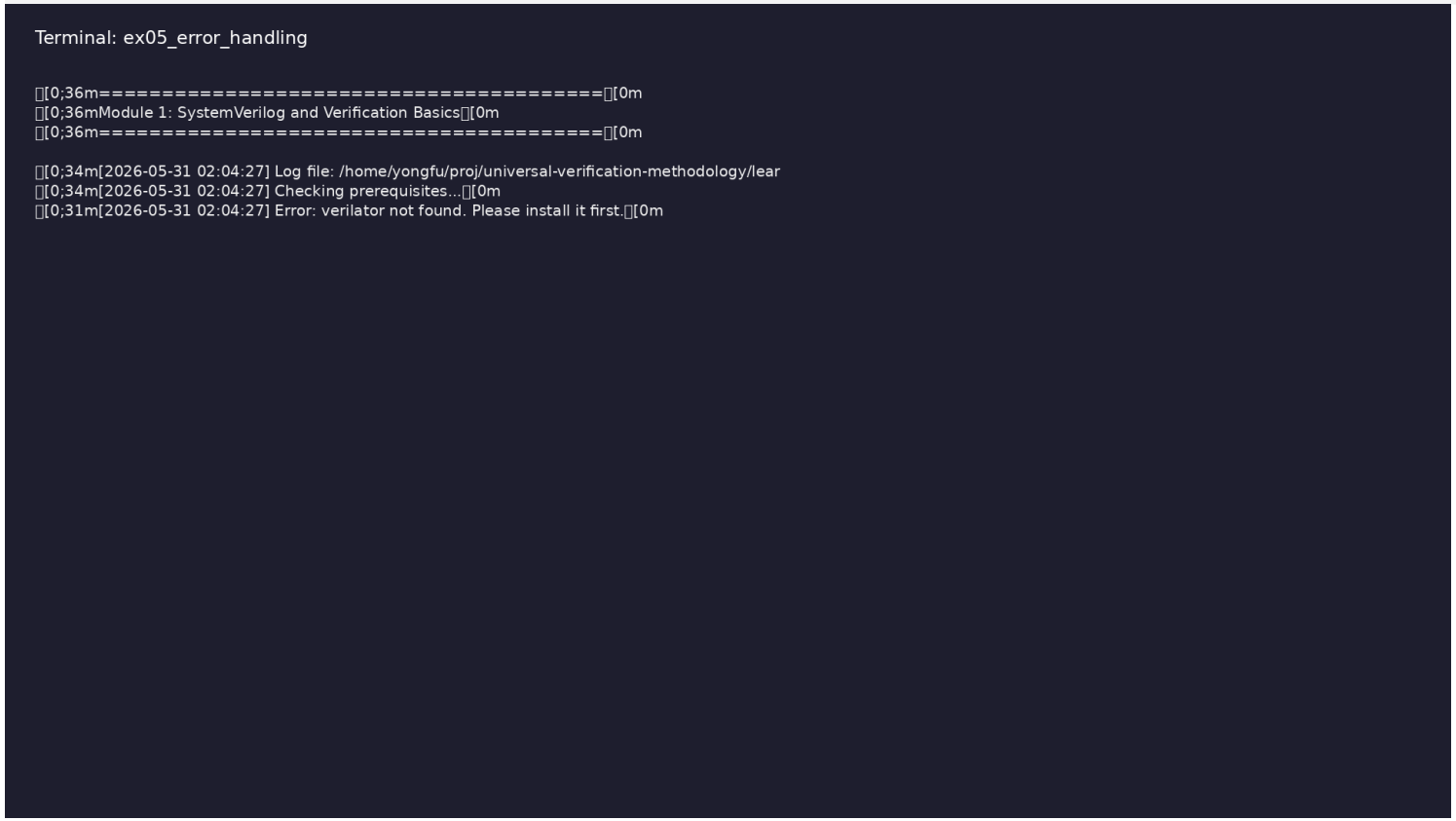


Example 5: Error Handling

- UVM Reporting: Using `uvm_report_info()`, `uvm_report_error()`, etc.
- Assertions: Immediate and concurrent assertions
- Error Handling: Try-catch blocks for exception handling
- Logging Patterns: Structured logging for verification
- Retry Logic: Task with retry mechanism using `retry_task()`

Demo: Error Handling

```
$ ./scripts/module1.sh --error-handling
```



Common pitfalls

Transaction Class Design

- Create a base transaction class
- Implement copy, compare, and string conversion methods
- Create derived transaction classes
- Test inheritance and polymorphism

Interface Design

- Design an interface for a simple bus protocol
- Create modports for master and slave
- Implement a clocking block
- Connect interface to a module

Package Organization

- Create a verification package
- Define common types and functions
- Import and use package in testbench
- Resolve name conflicts

Data Structure Usage

- Use dynamic arrays for transaction storage
- Implement a queue-based scoreboard
- Use associative arrays for coverage tracking
- Create structures for configuration

Error Handling

- Implement UVM reporting in testbench
- Add assertions to verify design behavior
- Handle exceptions gracefully
- Create structured logging

Practice & assessment

What you should know (1/2)

- Understand SystemVerilog OOP concepts
- Create and use SystemVerilog classes
- Design interfaces with modports
- Organize code using packages
- Use appropriate data structures for verification
- Implement error handling and logging

What you should know (2/2)

- Write basic SystemVerilog testbenches
- Understand UVM class structure basics

Exercises

- Transaction Class Design
- Interface Design
- Package Organization
- Data Structure Usage
- Error Handling

Assessment checklist

- Can create SystemVerilog classes with inheritance
- Can design and use interfaces with modports
- Can organize code using packages
- Can use appropriate data structures
- Can implement error handling and logging
- Can write basic SystemVerilog testbenches

Summary & next steps

- Understand SystemVerilog for verification and verification fundamentals
- Complete module1/CHECKLIST.md
- Review module1/EXAMPLES.md and run each lab
- Next: Next module in course